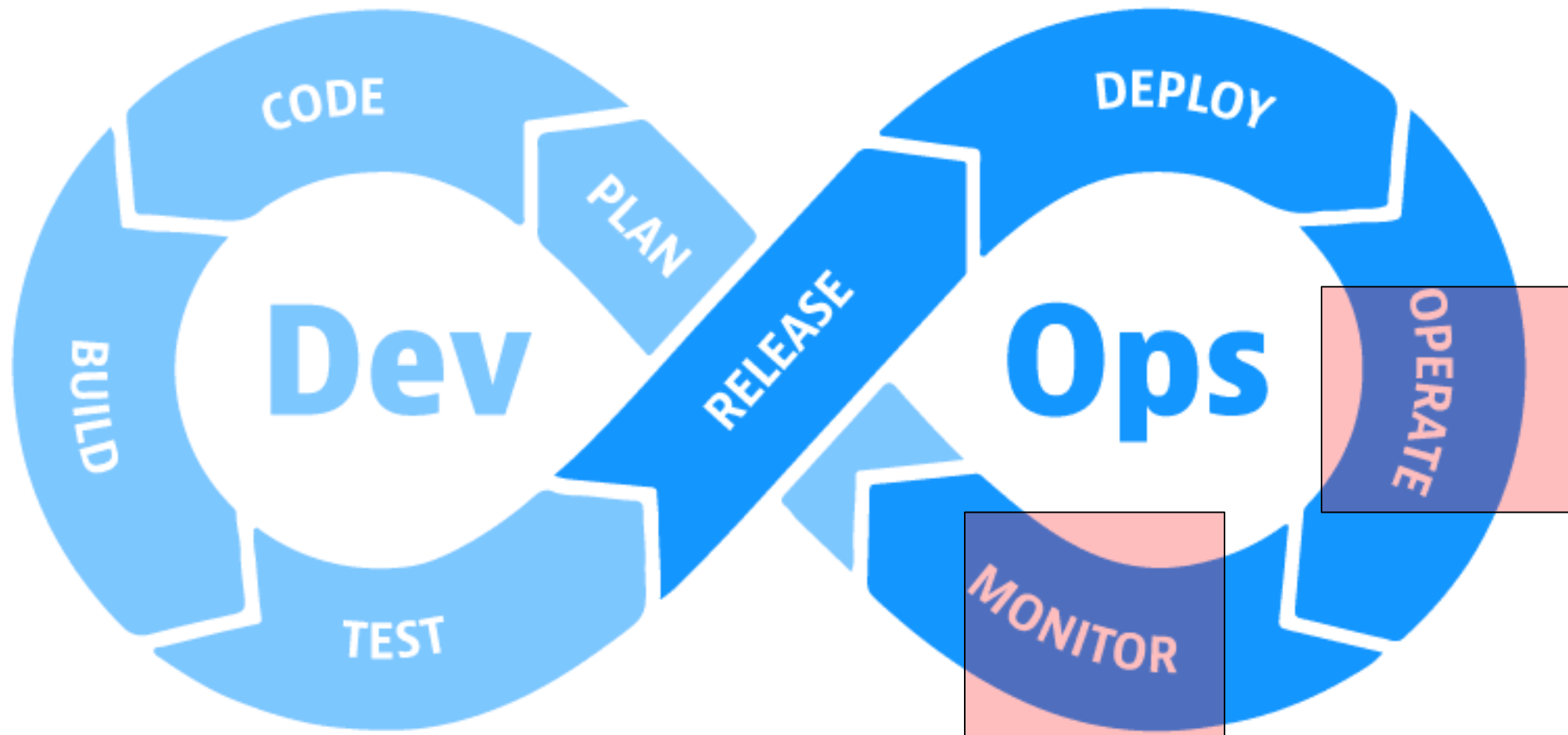


Developer Operations (devops)

Availability and SLOs



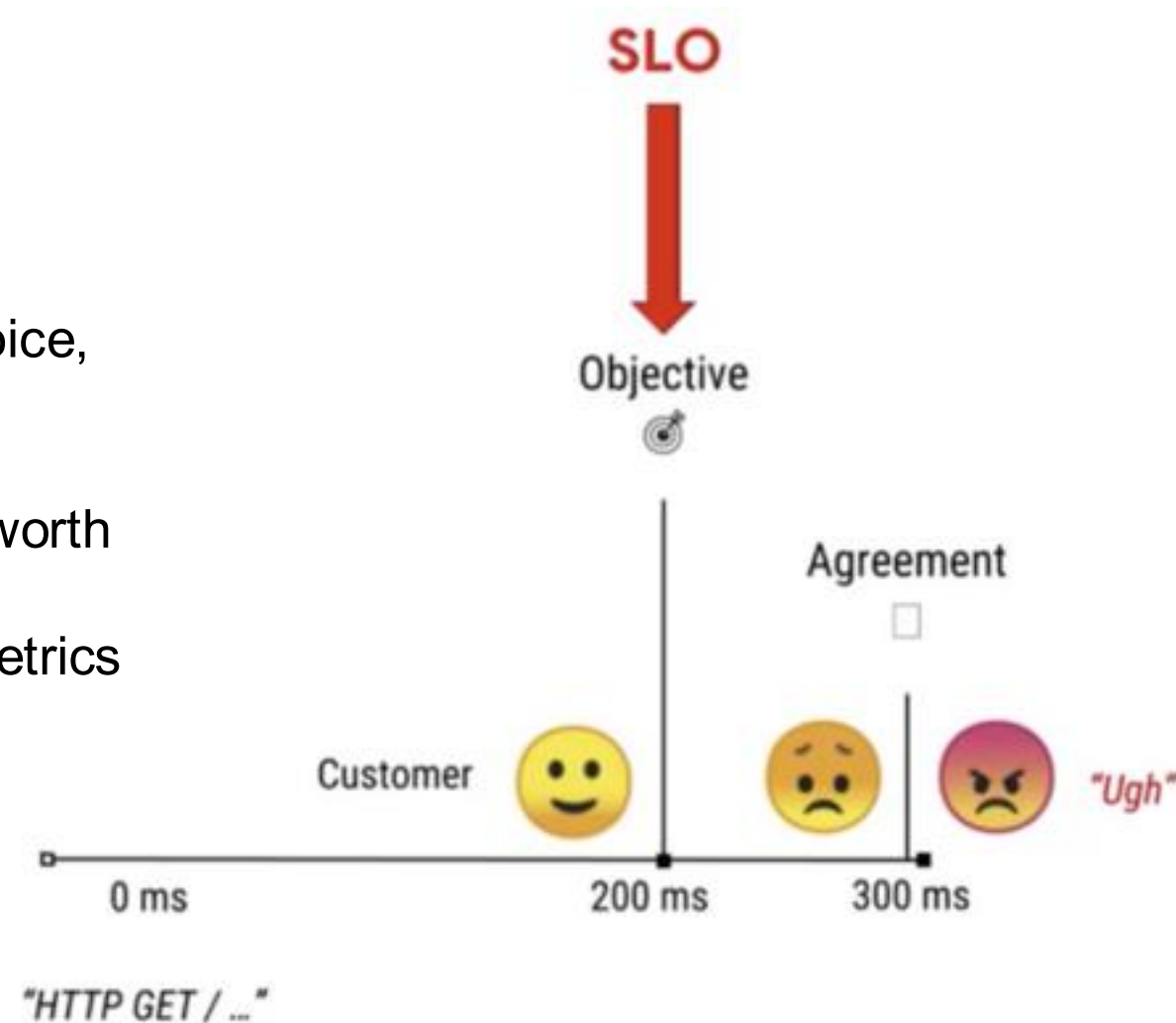
SLOs



Recap: Reliability is the most important feature

- If a system isn't reliable, users won't trust it.
- If users don't trust a system, when given a choice, they won't use it.
- Since all software systems are governed by network effects, if a system has no users, it's worth nothing.
- You are what you measure, so choose your metrics carefully.

→ <https://sre.google/workbook/reaching-beyond/>



What 99.999 really means...

		Downtime		
Availability %	Friendly Name	per Year	per Month	per Week
90%	one nine	36.5 days	72 hours	16.8 hours
99%	two nines	3.65 days	7.2 hours	1.68 hours
99.5%	--	1.83 days	3.6 hours	50.4 minutes
99.9%	three nines	8.76 hours	43.8 minutes	10.1 minutes
99.95%	--	4.38 hours	21.56 minutes	5.04 minutes
99.99%	four nines	52.56 minutes	4.32 minutes	1.01 minutes
99.999%	five nines	5.26 minutes	25.9 seconds	6.05 seconds
99.9999%	six nines	31.5 seconds	2.59 seconds	0.605 seconds
99.99999%	seven nines	3.15 seconds	0.259 seconds	0.0605 seconds

Want VS Need

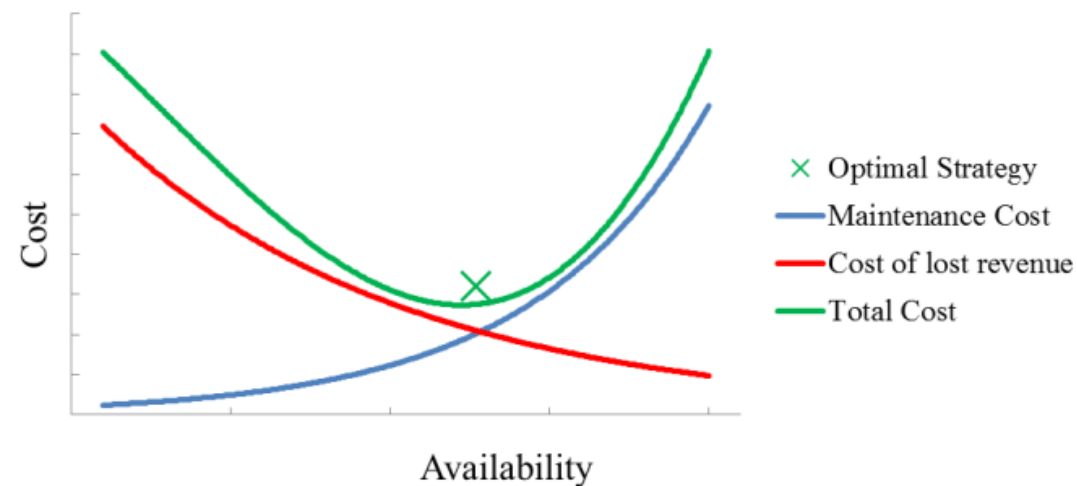
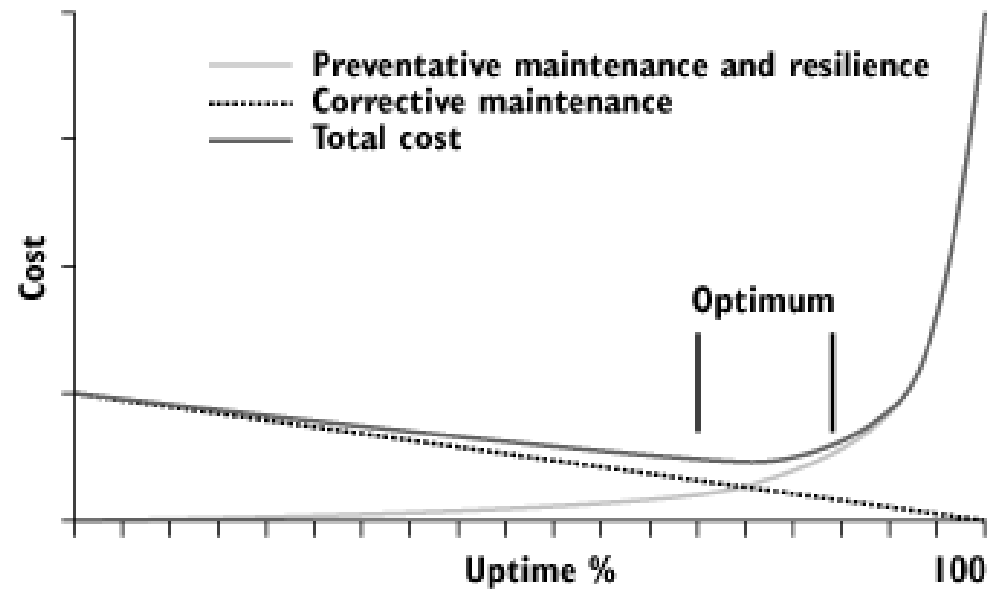
We all want 100% availability:

- Do we need it?
- Do we want to pay for it?
- How much are we willing to pay for?
- 99.999% is rooted in want not need

→ Costs increase exponentially with higher availability

→ Technical Debt is main driver for lower availability

→ Selling higher availability != Expectation of higher availability



99.999% really possible?

According to ITIC Survey, 44% of outages are based on Human Error:

Can those be fixed in under 5mins?

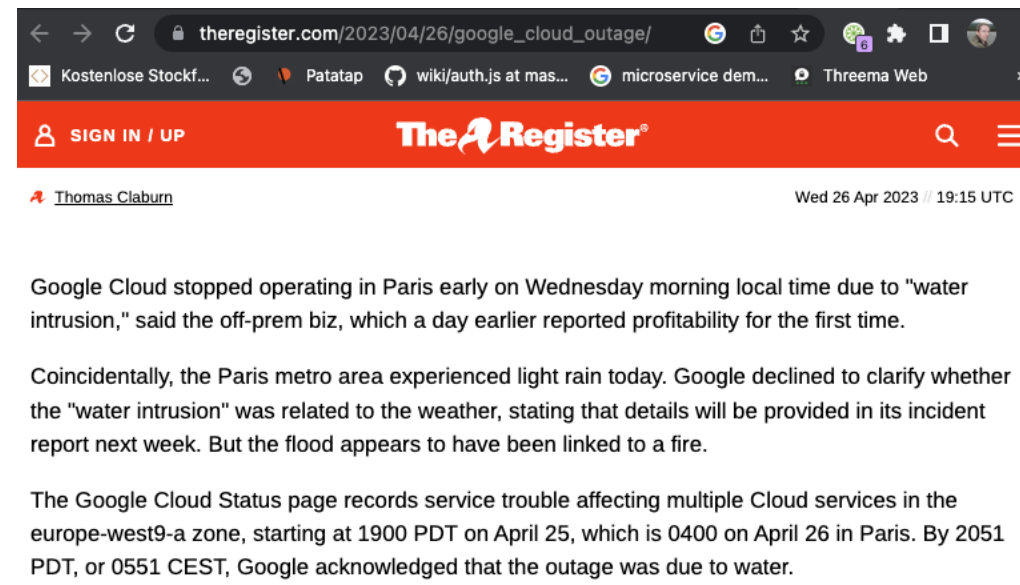
One wrong parameter brought S3 down in 2017 for 4 hours
(<https://aws.amazon.com/message/41926/>)

1. The connection to the services must be counted in as well:
Is the internet connection 99.999% reliable?

2. How available is my platform?
Compare Providers with Hyperscaler

3. Incidents happen...
“multi-cluster failure [...] led to an emergency shutdown of multiple zones”

Application / Service	SLA
AWS Single Availability Zone	99.95
AWS Region (Multi-AZ)	99.99
AWS S3	99.9
Google Compute Engine (GCE)	99.95
Google Apps for Work	99.9
Azure	99.9



Green.ch: 99.9%
Swisscom: ~99.3%

Availability, Sequential Availability

Series Components = $C_1 * C_2 * C_3 * C_4 \dots$

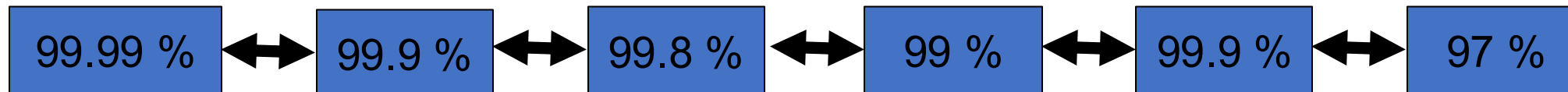
Overall Availability



25 %



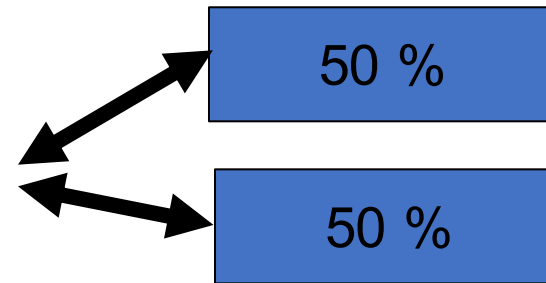
99.8%



95%

Availability, Parallel Availabilty

$$\text{Parallel Component} = 1 - (1 - C_x)^2$$

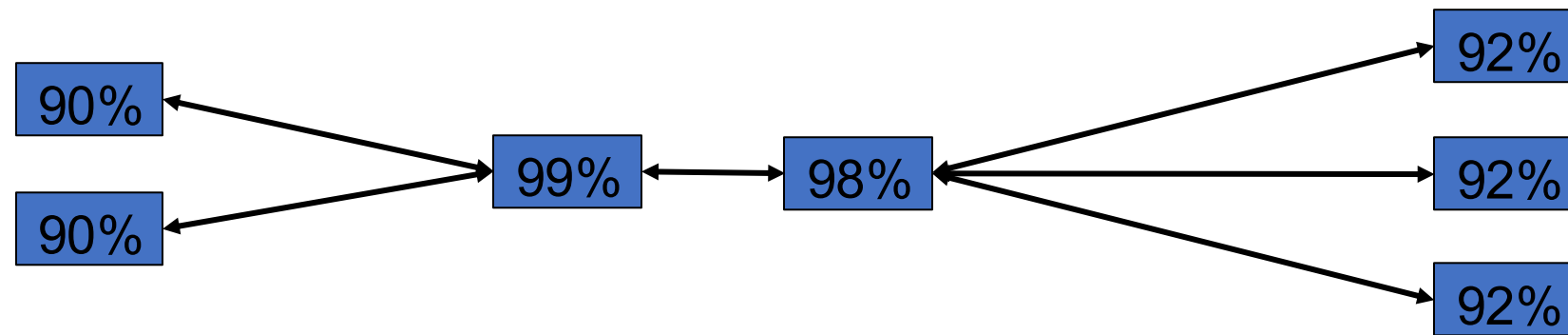


75 %



99.9999%

Chaining of Availability

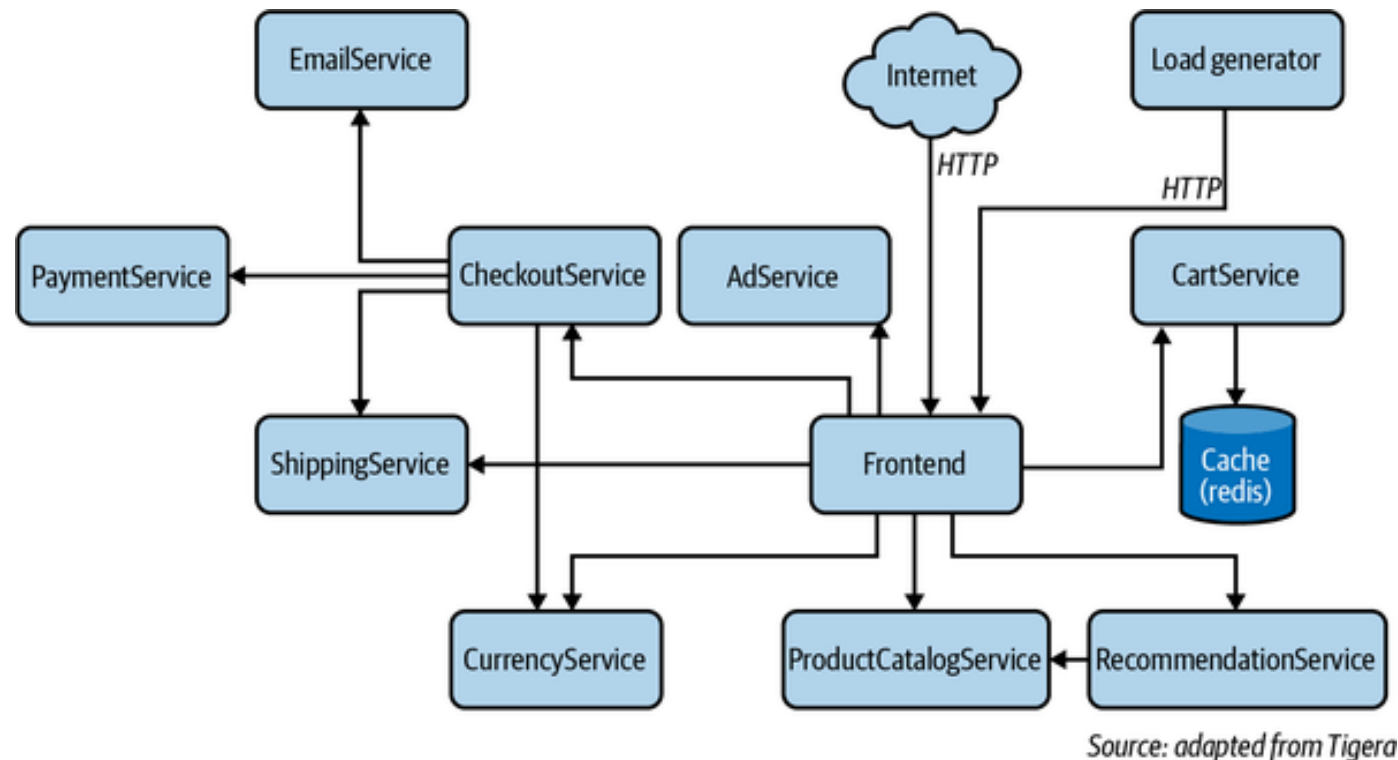


$$1 - (1 - 0.9)^2 = 0.99$$

$$1 - (1 - 0.92)^3 = 0.999$$

$$0.99 * 0.99 * 0.98 * 0.999 = 0.960$$

Availability, Parallel vs Serial



Parallel:

- HA-Setup of same services
- E.g. Horizontal Scaling

Serial:

- Different Services of different kinds
- E.g. Binding to database and messaging and Load Balancer

99.999% is expensive

Redundancy related to app

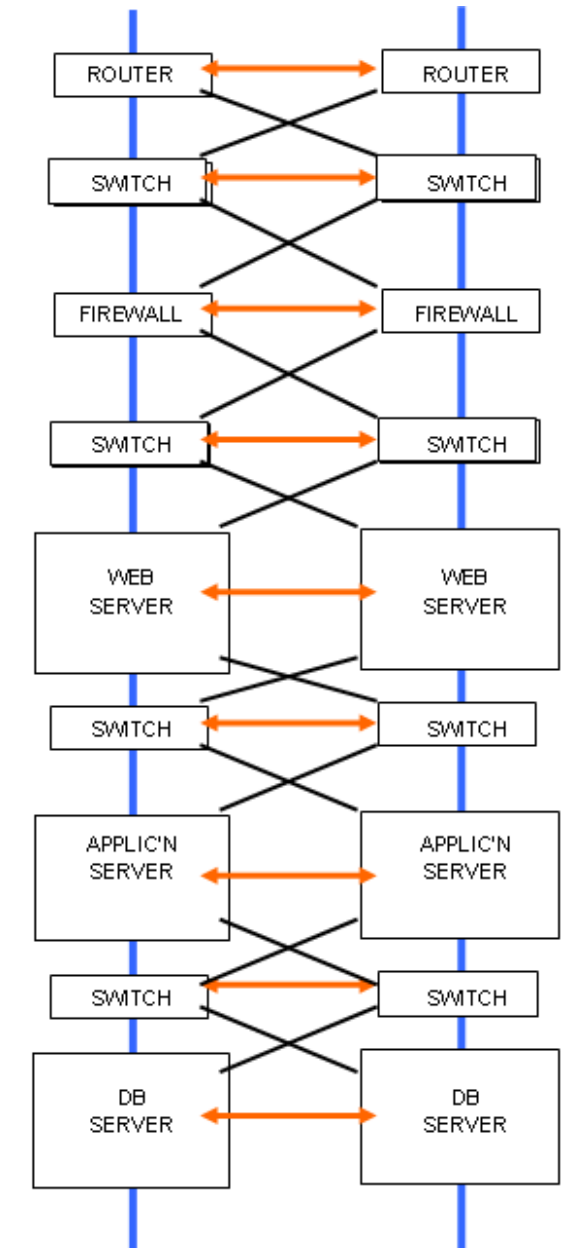
- Horizontal Scaling
- Beware the state of an app
- Beware the CAP-Theorem

Redundancy related to platform

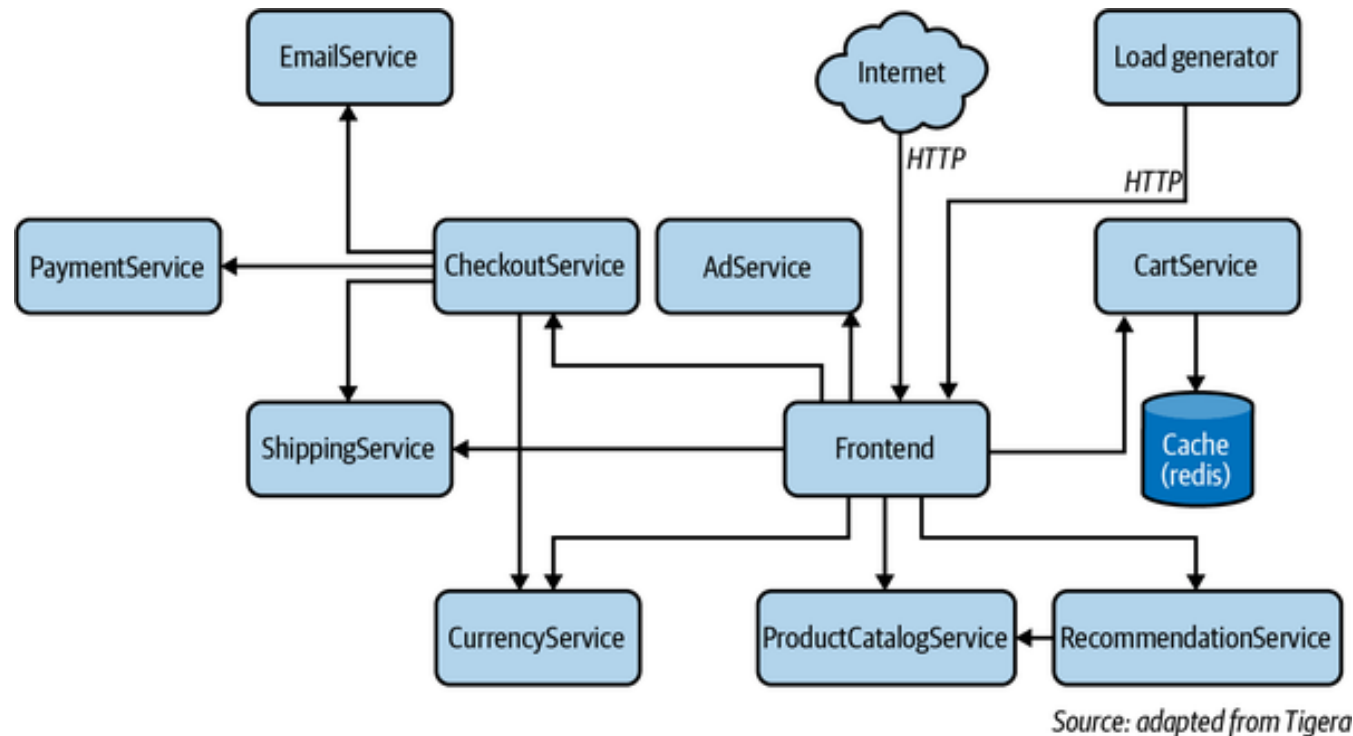
- Do not underestimate scaling application server layers
- See the Kubernetes-Tutorials in our case (and this is playground!)

Redundancy related to computer center / multi region

- You have to rely on Infrastructure as Code
- Beware of latency issues



Caveats of common Availability Assumptions



Plain sequential availability

- If only the plain overall availability is taken into account, the overall availability is not this good:

- 12 components
- All Sequential
- Assume 99% per component: 88% overall
- Hard to increase in a distributed environment

→ Caveat 1:

Finer granular perspective: per service
(is RecommendationService always needed like ShippingService?)

SLA?

Follow Up question:
What is availability?

- Is the service accessible via ping?
- How fast is the access?
- How fast is the transfer?
- Of what value is the access if no login/storage/data retrieval is possible?

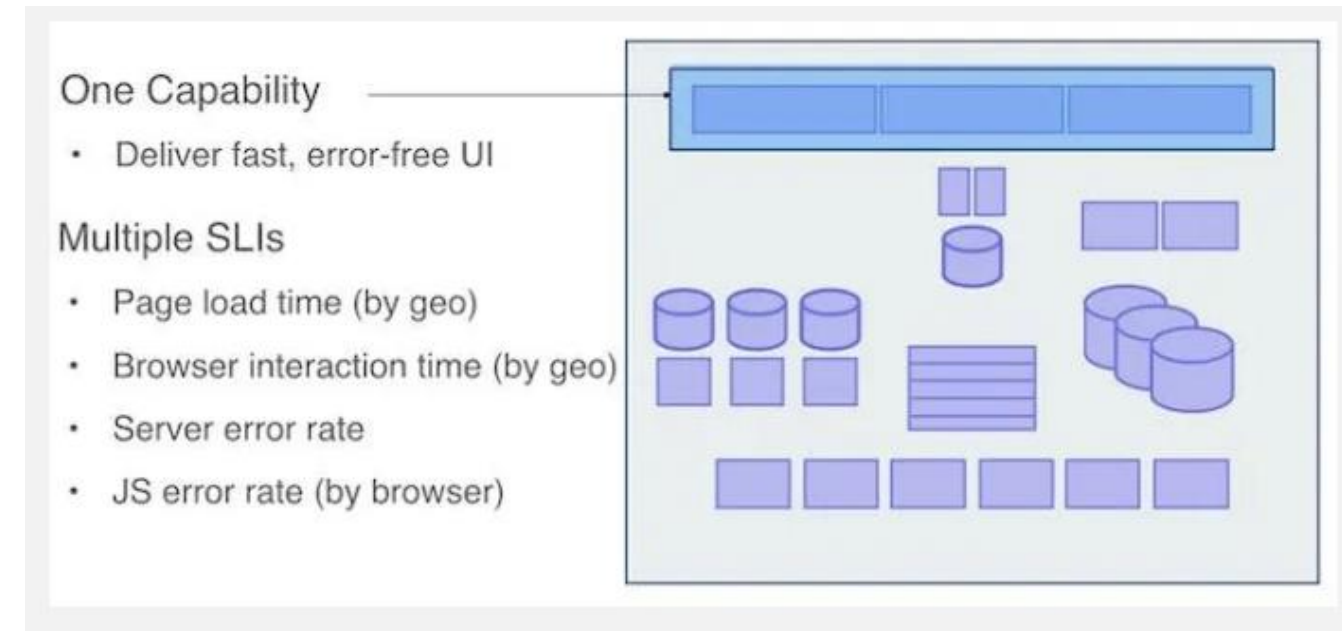
→ Caveat 2: The term SLA is not defined for concrete technical operations. So availability can not be defined that way



SLI

Service Level Indicator

- Base for defining availability
 - For one specific action/attribute
 - Of one specific service
- Examples to be defined
 - Golden Signals of one specific service of one operation (the concise the better) for network service
 - Durability for storage
 - Correctness for computation
- Just the metrics, no thresholds / rules to meet
- Need to be derived automatically



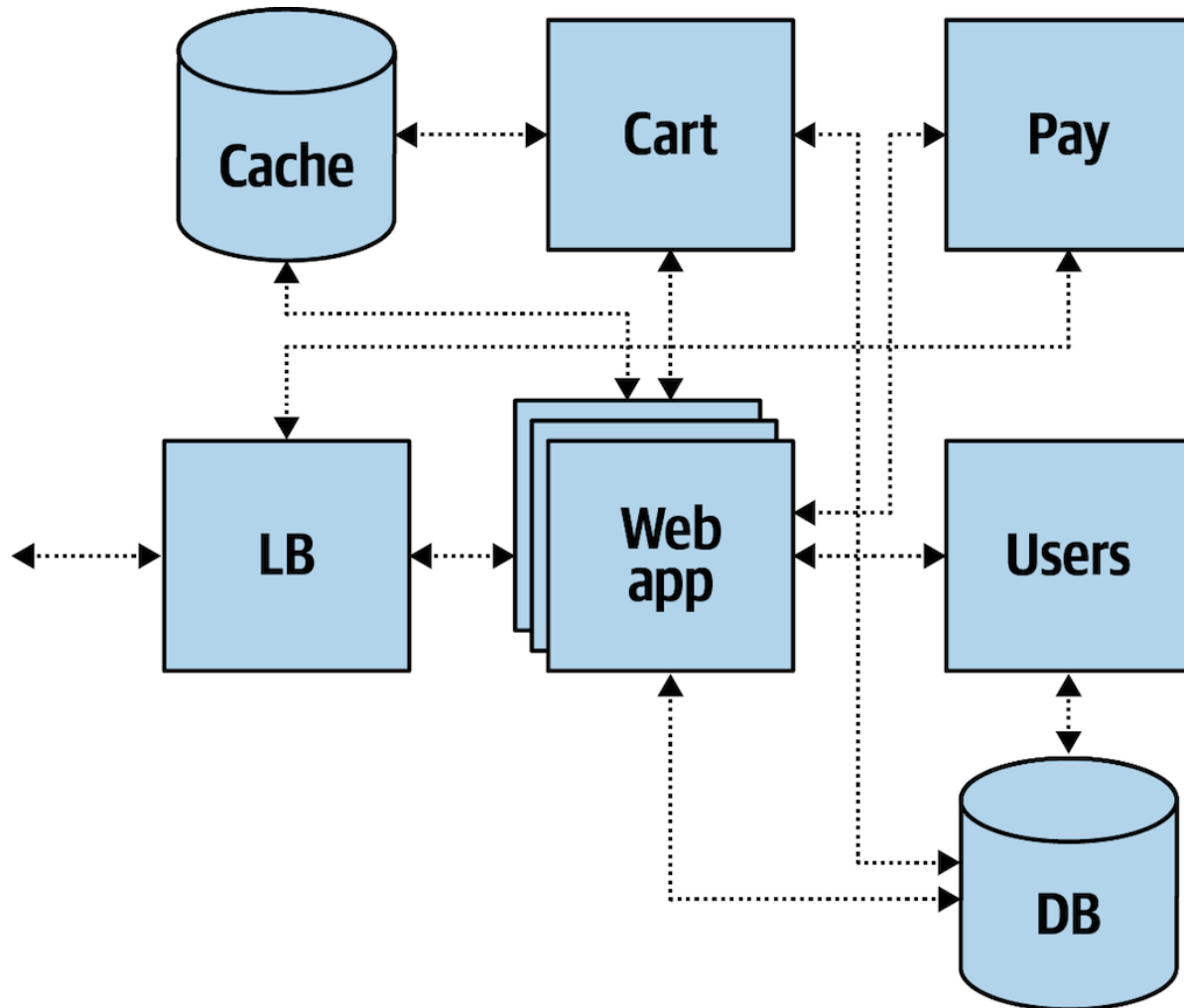
$$SLI = \left(\frac{\text{good events}}{\text{valid events}} \right) \times 100$$

SLO

Service Level Objective

- Threshold to be hold for defined SLIs:
 $SLI \leq \text{target threshold}$
- Technically hard to define, need to be refined
 - Wrong SLI $\rightarrow$ no use
 - Threshold too low $\rightarrow$ Customer/Services affected
 - Threshold too high $\rightarrow$ Too many incidents, false alarms
 - Must be simple yet holistic
 - Avoid absolutes (always available, for all data accesses, etc.)
- Organizational hard to develop
 - Must be defined with product management
 - Have as few SLOs as possible (but as many as necessary)

Target for SLOs



Interaction	Components
Visit home page	Load balancers, web app, cache
Browse items	Load balancers, web app, database
Add/remove cart item	Load balancers, web app, cart service, cache, database
Edit shipping address	Load balancers, web app, user service, database
Purchase item	Load balancers, web app, payment gateway, third-party payment vendor

Example for SLOs

1. $p95(\text{http_latency}[\text{path}=\text{webappl/impressum}]) < 50$
95 % of all http requests the impressum must be solved under 50ms
2. $p99(\text{http_latency}[\text{path}=\text{webappl/impressum}]) < 70$
99 % of all http requests the impressum must be solved under 70ms
3. $\text{sum}(\text{http_duration}\{\text{path}=\text{webapp/succeedCheckout}, \text{status} \sim "2.."\}) / \text{sum}(\text{http_duration}\{\text{path}=\text{webapp/Checkout}\}) > 0.99$
99% of all checkout requests are successful
4. $\text{rate}(\text{sum}(\text{pod_info}\{\text{app}=\text{webapp}\})) > 2$
Number of pods for webapp should always be redundant

SLA

Service Level Agreement

- Consequence if SLO is not met
- Legal and easy language with fixed defined consequences
- Promise against Customer defined by Product Management (not DevOps any more)
- Not of interest for the rest of the lecture since not definable by sourcecode but in contracts

Putting it all together

SLI:

What are we measuring?

E.g. How much time take the search results

SLO:

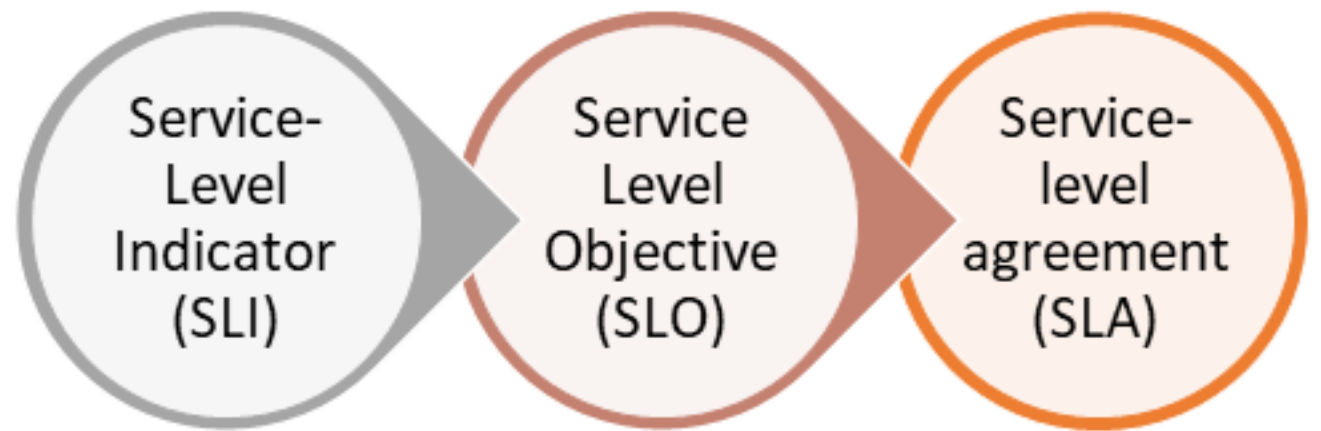
How well do we perform on the SLI?

E.g. Queries should be return results within 500ms

SLA:

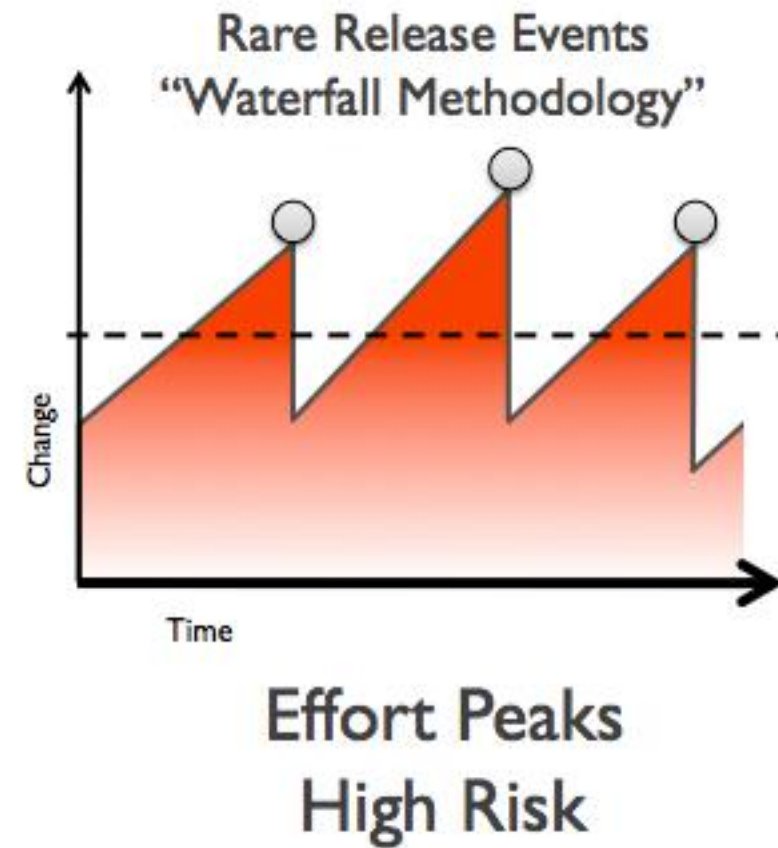
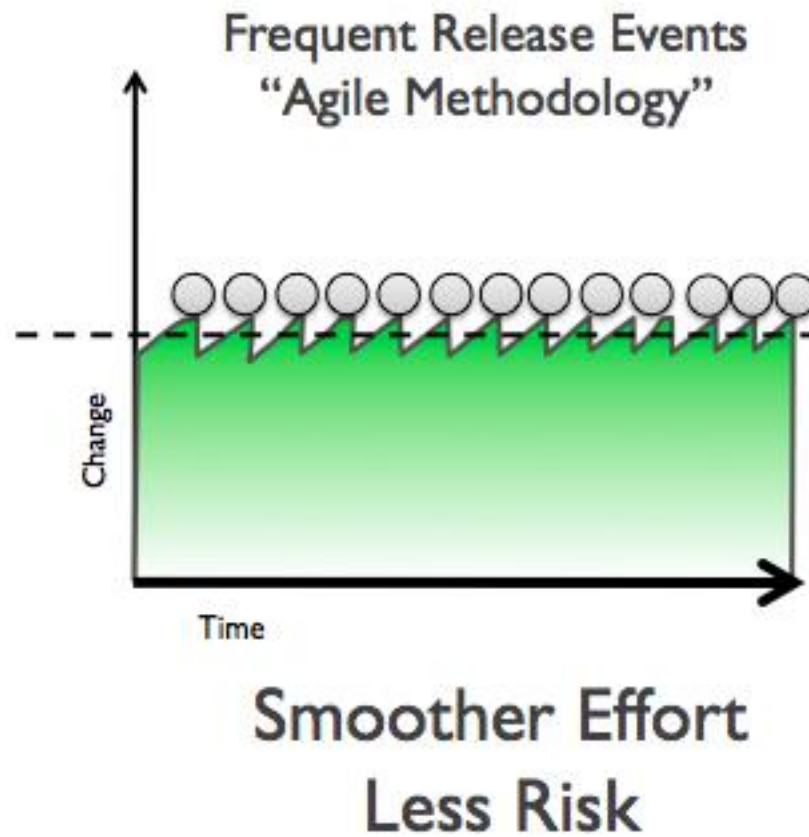
Consequences for missing objectives

E.g. Apologize, payback, ...



	SLIs	SLOs	SLAs
What they are:	Indicators of system health based on user priorities	Level of an SLI that corresponds to when users are happy with the service	Guarantee between the org and users promising some level of availability
How they help:	Highlights user priorities for your service	Prevents SLO breaches and provides an error budget	Builds customer confidence
Who builds them:	DevOps and SRE teams, in collaboration with customer-facing teams	DevOps and SRE teams, in collaboration with customer-facing teams	Customer-facing teams, in collaboration with legal teams
What happens if they're breached	N/A	Code freezes and other preventative measures to prevent SLA breach	Consequences set out in the SLA, possibly including reimbursement or credits

Recap: Accelerate the flow

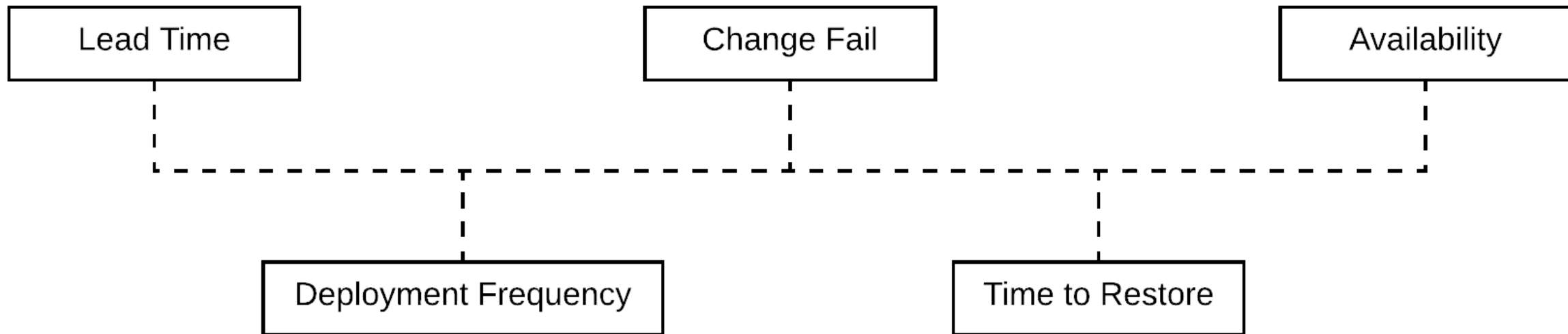


Dora Metrics

Software Development

Software Deployment

Service Operation



How can the optimization against different metrics be balanced?

Error Budgets

Newtons Law of Motion applied:

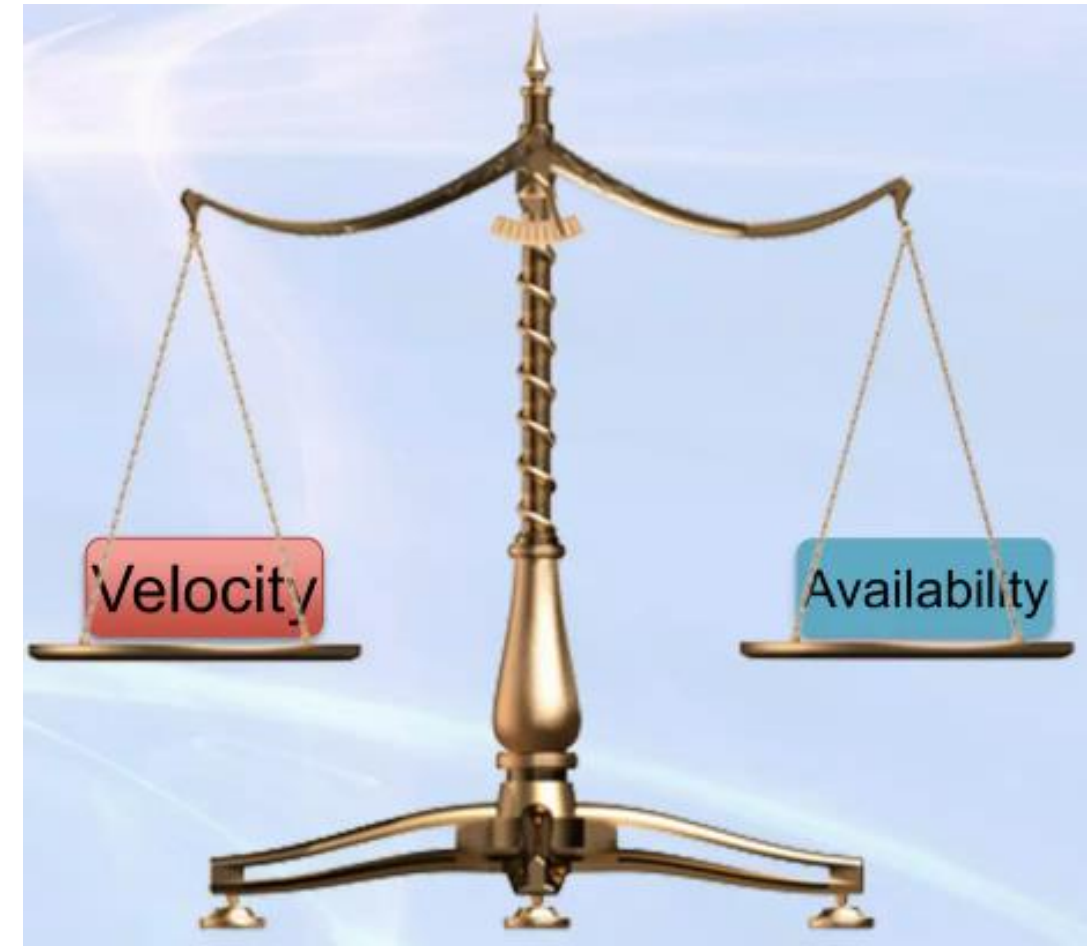
1: Objects at rest tend to stay at rest until an outside force is applied.

→ The most stable systems are the ones never touched.

2: Momentum change is proportional to the amount of force applied.

→ The more changes an application receives, the higher the likelihood errors will be introduced.

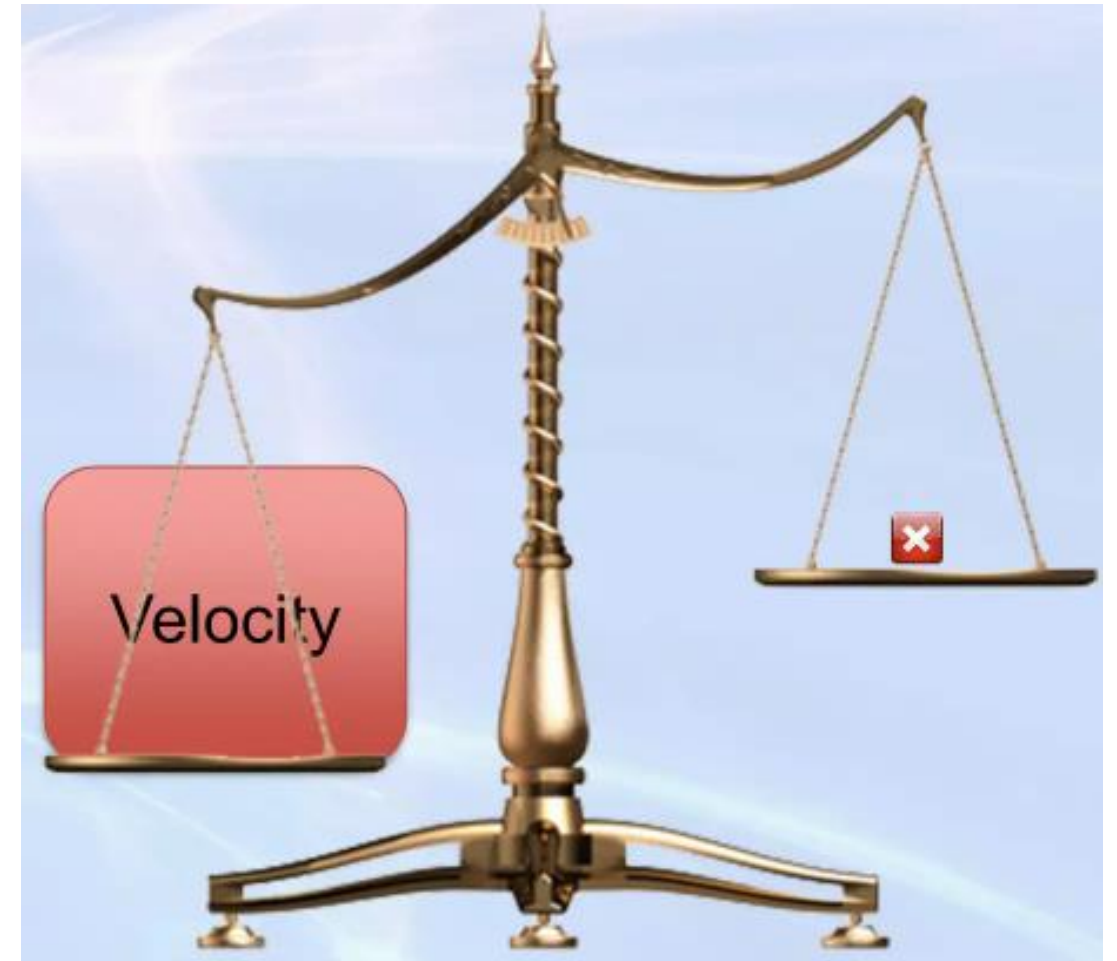
→ SLOs to the rescue: All errors within the SLOs are "legit" and within the Error Budget.



Error Budgets not spent

When within the budget, increase velocity, take more risk

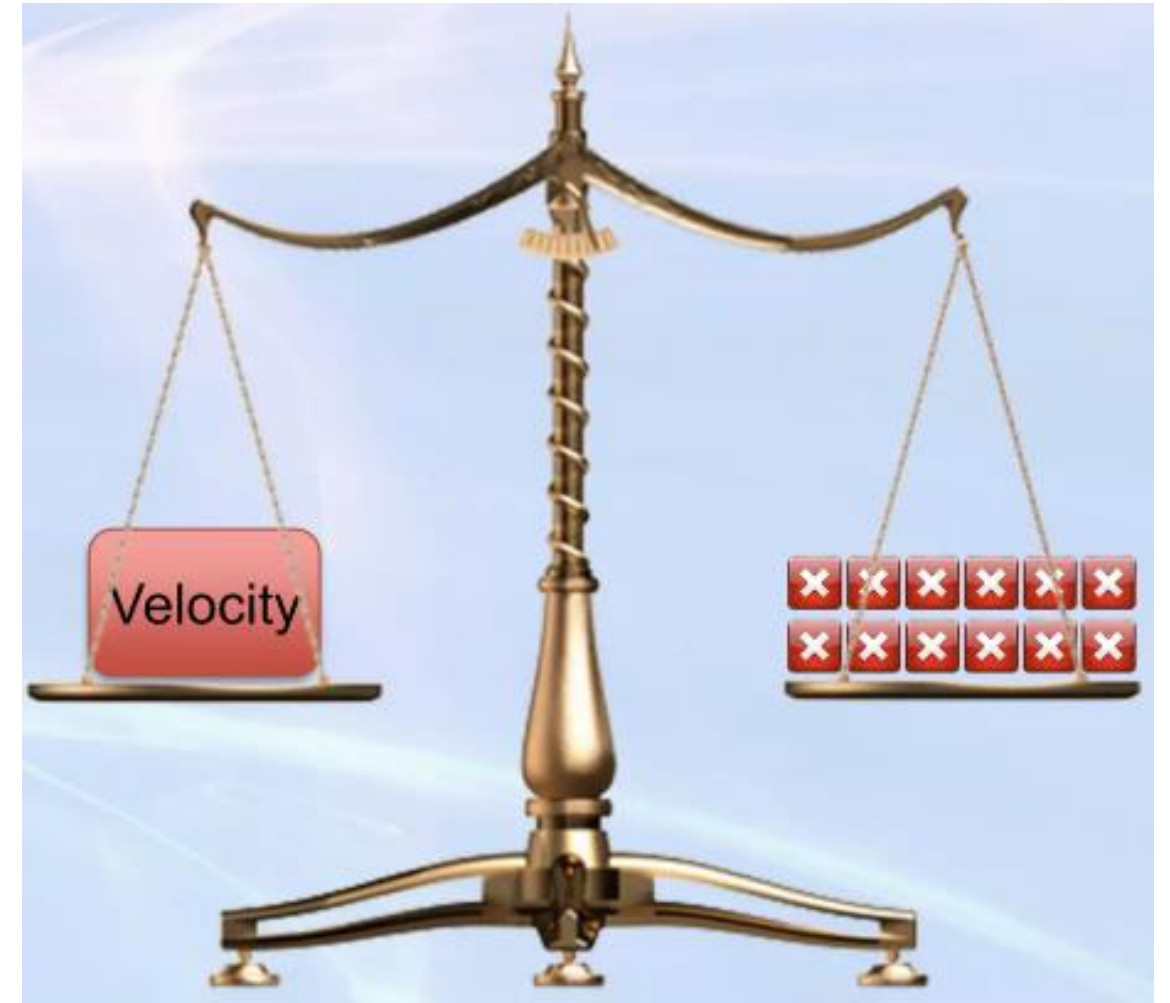
- More features
- Less QA
- Larger Canaries
- Move Faster



Error Budgets

When budget is spent, decrease velocity

- Less Features
- More QA
- Smaller Canaries
- Move Slower



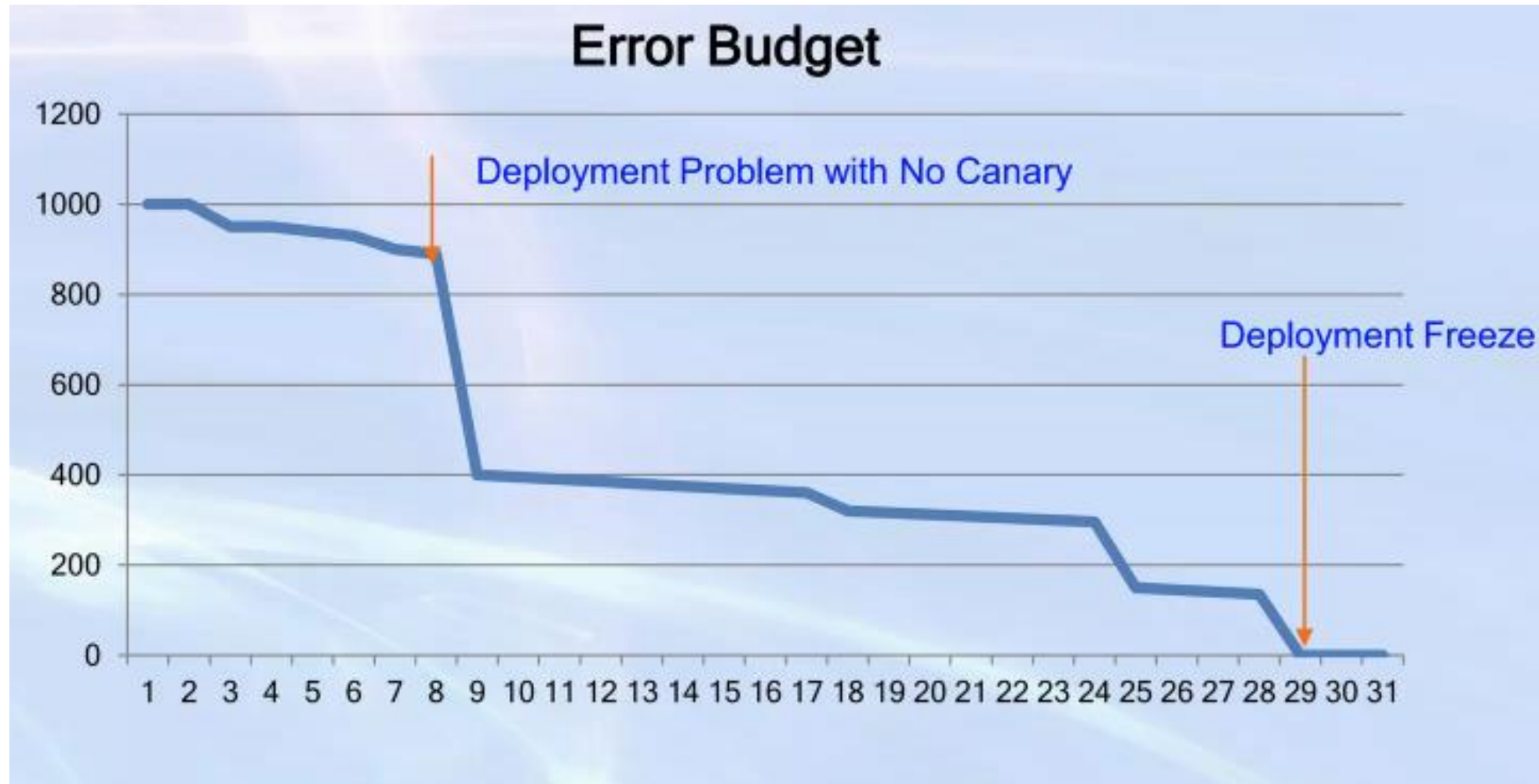
Error Budgets

If budget is totally consumed, stop deployments

- Analysis why budget was spent
- Reduce technical debt
- Invest in tooling, maintaining
- No new Feature
- Stop



Error Budgets



SLOs and Upper Bound

Chubby [\[Bur06\]](#) is Google's lock service for loosely coupled distributed systems. In the global case, we distribute Chubby instances such that each replica is in a different geographical region. Over time, we found that the failures of the global instance of Chubby consistently generated service outages, many of which were visible to end users. As it turns out, true global Chubby outages are so infrequent that service owners began to add dependencies to Chubby assuming that it would never go down. Its high reliability provided a false sense of security because the services could not function appropriately when Chubby was unavailable, however rarely that occurred.

Best practices for SLIs/SLOs

- Identify your metrics
 - Learn your application
 - Identify SLIs and define SLOs on this behaviour and on external needs
- Define SLIs and SLOs for specific capabilities at system boundaries
 - Remember the 4 causes for failure (Internal System Changes, Changes in User Behaviour, Changes in dependencies and Changes in platform) → those are system boundaries
- Combine multiple SLIs in one SLO
- Document and Share your SLIs/SLOs
 - Transparent Dashboards, Rules in Sourcecode
- Get Management Backing
 - Especially when working with Error Budgets

